Prefix Sums and Their Applications

February 13, 2026

1 Introduction

Definition 1.1. The **all-prefix-sums** operation takes a binary associative operator $\oplus$, and an ordered set of n elements

$$[a_0, a_1, \dots, a_{n-1}]$$

and returns the ordered set

$$[a_0, (a_0 \oplus a_1), \dots, (a_0 \oplus a_2 \oplus \cdots \oplus a_{n-1})]$$

Uses:

1. To lexically compare strings of characters. For example, to determine that `strategy` should appear before `stratification` in a dictionary
2. To add multi precision numbers. These are numbers that cannot be represented in a single machine word.
3. To evaluate polynomials
4. To solve recurrences. For example, to solve the recurrences $x_i = a_i x_{i-1} + b_i x_{i-2}$ and $x_i = a_i + b_i / x_{i-1}$
5. To implement radix sort
6. To implement quicksort
7. To solve tridiagonal linear systems
8. To delete marked elements from an array
9. To dynamically allocate processors

10. To perform lexical analysis. For example, to parse a program into tokens.

11. To search for regular expressions. For example, to implement the UNIX grep program.

12. To implement some tree operations. For example, to find the depth of every vertex in a tree

13. To label components in two dimensional images.

```
proc all-prefix-sums(Out, In)
  i ← 0
  sum ← In[0]
  Out[0] ← sum
  while (i < length)
    i ← i + 1
    sum ← sum + In[i]
    Out[i] ← sum
```

Vector Version

```
proc all-prefix-sums(Out, In)
  i ← 0
  sum ← In[0].value
  Out[0] ← sum
  while (In[i].pointer ≠ EOL)
    i ← In[i].pointer
    sum ← sum + In[i].value
    Out[i] ← sum
```

List Version

FIGURE 1.1
Sequential algorithms for calculating the all-prefix-sums operation with operator + on a vector and on a linked-list. In the list version, each element of `In` consists of two fields: a value (`.value`), and a pointer to the next position in the list (`.pointer`). EOL means the end-of-list pointer.

Figure 1

Definition 1.2. The **scan** operation is a vector all-prefix-sums operation

Definition 1.3. The **prescan** operation takes a binary associative operator $\oplus$ with identity I, and a vector of n elements

$$[a_0, \dots, a_{n-1}]$$

and returns the vector

$$[I, a_0, (a_0 \oplus a_1), \dots, (a_0 \oplus a_1 \oplus \cdots \oplus a_{n-2})]$$

2 Implementation

This section describes an algorithm for calculating the scan operation in parallel. For p processors and a vector of length n on an EREW PRAM, the algorithm has a time complexity of $O(n/p + \log p)$. The algorithm is simple

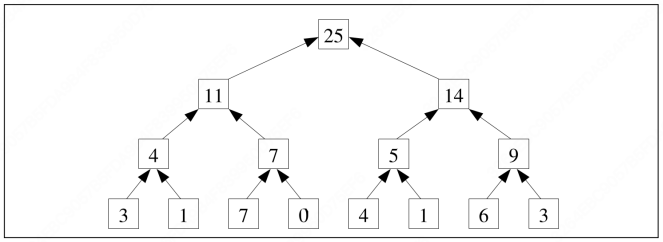
and well suited for direct implementation in hardware. Chapter 4 shows how the time of the scan operation with certain operators can be reduced to $O(n/p + \log p / \log\log p)$ on a CREW PRAM.

Before describing the scan operation, we consider a simpler problem, that of generating only the final element of the scan. We call this the **reduce** operation.

Definition 2.1. The **reduce** operation takes a binary associative operator $\oplus$ with identity i, and an ordered set $[a_0, \ldots, a_{n-1}]$ of n elements, and returns the value $a_0 \oplus \cdots \oplus a_{n-1}$

A balanced binary tree can be used to implement the reduce operation by laying the tree over the values, and using $\oplus$ to sum pairs at each vertex (see Figure **??**a). The correctness of the result relies on $\oplus$ being associative. The operator, however, does not need to be commutative since the order of the operands is maintained.

On an EREW PRAM, each level of the tree can be executed in parallel, so the implementation can step from the leaves to the root of the tree (see Figure **??**b); we call this an **up-sweep**. Since the tree is of depth $\lceil \log n \rceil$, and one processor is needed for every pair of elements, the algorithm requires $O(\log n)$ time and $n/2$ processors.



(a) Executing a +-reduce on a tree.

for d **from** 0 **to** $(\lg n) - 1$
  in parallel for i **from** 0 **to** $n-1$ **by** 2^{d+1}
    $a[i + 2^{d+1} - 1] \leftarrow a[i + 2^d - 1] + a[i + 2^{d+1} - 1]$

Step	Vector in Memory							
0	[$\boxed{3}$	$\boxed{1}$	$\boxed{7}$	$\boxed{0}$	$\boxed{4}$	$\boxed{1}$	$\boxed{6}$	$\boxed{3}$]
1	[3	$\boxed{4}$	7	$\boxed{7}$	4	$\boxed{5}$	6	$\boxed{9}$]
2	[3	4	7	$\boxed{11}$	4	5	6	$\boxed{14}$]
3	[3	4	7	11	4	5	6	$\boxed{25}$]

(b) Executing a +-reduce on a PRAM.

FIGURE 1.2
An example of the reduce operation when $\oplus$ is integer addition. The boxes in (b) show the locations that are modified on each step. The length of the vector is n and must be a power of two. The final result will reside in $a[n-1]$.

Figure 2

in parallel for each processor i
  `sum`$[i] \leftarrow a[(n/p)i]$
  `for` j `from` 1 `to` n/p
    `sum`$[i] \leftarrow$ `sum`$[i]$ `+` $a[(n/p)i + j]$
`result` $\leftarrow$ `+-reduce(sum)`

$$[\underbrace{4 \quad 7 \quad 1}_{\text{processor 0}} \quad \underbrace{0 \quad 5 \quad 2}_{\text{processor 1}} \quad \underbrace{6 \quad 4 \quad 8}_{\text{processor 2}} \quad \underbrace{1 \quad 9 \quad 5}_{\text{processor 3}}]$$

Processor Sums	=	[12	7	18	15]
Total Sum	=	52			

FIGURE 1.3
The +-reduce operation with more elements than processors. We assume that n/p is an integer.

Figure 3

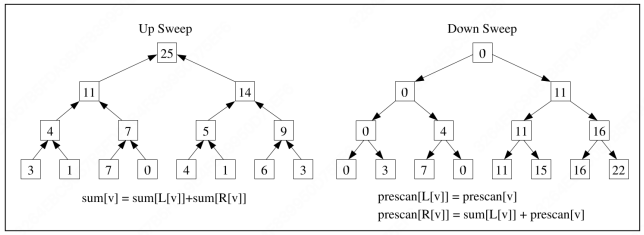
If we assume a fixed number of processors p, with $n > p$, then each processor can sum an n/p section of the vector to generate a processor sum;

the tree technique can then be used to reduce the processor sums (see Figure **??**). The time taken to generate the processor sums is $\lceil n/p \rceil$, so the total time required on an EREW PRAM is:

$$T_R(n,p) = \lceil n/p \rceil + \lceil \log p \rceil = O(n/p + \log p)$$

When $n/p \geq \log p$ the complexity $O(n/p)$.

We now return to the scan operation. We actually show how to implement the prescan operation; the scan is then determined by shifting the result and putting the sum at the end. If we look at the tree generated by the reduce operation, it contains many partial sums over regions of the vector. It turns out that these partial sums can be used to generate all the prefix sums. This requires executing another sweep of the tree with one step per level, but this time starting at the root and going to the leaves (a down-sweep). Initially, the identity element is inserted at the root of the tree. On each step, each vertex at the current level passes to its left child its own value, and it passes to its right child, $\oplus$ applied to the value from the left child from the up-sweep and its own value



(a) Executing a +-prescan on a tree.

```
procedure down-sweep(A)
  a[n − 1] ← 0                                  % Set the identity
  for d from (lg n) − 1 downto 0
    in parallel for i from 0 to n − 1 by 2^(d+1)
      t ← a[i + 2^d − 1]                        % Save in temporary
      a[i + 2^d − 1] ← a[i + 2^(d+1) − 1]       % Set left child
      a[i + 2^(d+1) − 1] ← t + a[i + 2^(d+1) − 1]  % Set right child
```

	Step	Vector in Memory							
	0	[$\boxed{3}$	$\boxed{1}$	$\boxed{7}$	$\boxed{0}$	$\boxed{4}$	$\boxed{1}$	$\boxed{6}$	$\boxed{3}$]
up	1	[3	$\boxed{4}$	7	$\boxed{7}$	4	$\boxed{5}$	6	$\boxed{9}$]
	2	[3	4	7	$\boxed{11}$	4	5	6	$\boxed{14}$]
	3	[3	4	7	11	4	5	6	$\boxed{25}$]
clear	4	[3	4	7	11	4	5	6	$\boxed{0}$]
down	5	[3	4	7	$\boxed{0}$	4	5	6	$\boxed{11}$]
	6	[3	$\boxed{0}$	7	$\boxed{4}$	4	$\boxed{11}$	6	$\boxed{16}$]
	7	[$\boxed{0}$	$\boxed{3}$	$\boxed{4}$	$\boxed{11}$	$\boxed{11}$	$\boxed{15}$	$\boxed{16}$	$\boxed{22}$]

(b) Executing a +-prescan on a PRAM.

FIGURE 1.4
A parallel prescan on a tree using integer addition as the associative operator $\oplus$, and 0 as the identity.

Figure 4

Let's consider why the down-sweep works. We say that vertex x **precedes** vertex y if x appears before y in the preorder (root, left, right) traversal of the tree

Theorem 2.2. *After a complete down-sweep, each vertex of the tree contains the sum of all the leaf values that precedes it*

Proof. Induction on the root: if a parent has the correct sum, both children must have the correct sum. The root has no element preceding it, so its value is correctly the identity element.

Consider Figure **??**. The left child of any vertex has exactly the same leaves preceding it as the vertex itself (the leaves in region A in the figure).

This is because the preorder traversal always visits the left child of a vertex immediately after the vertex. By the induction hypothesis, the parent has the correct sum, so it need only copy this sum to the left child.

The right child of any vertex has two sets of leaves preceding it, the leaves preceding the parent (region A), and the leaves at or below the left child (region B). Therefore, by adding the parent's down-sweep value, which is correct by the induction hypothesis, and the left-child's up-sweep value, the right-child will contain the sum of all the leaves preceding it. □

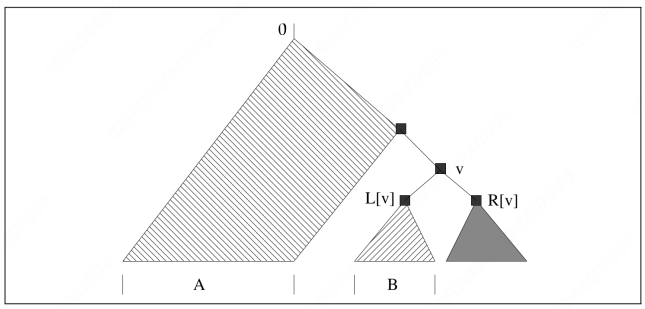


FIGURE 1.5
Illustration for Theorem 1.1.

Figure 5

On an EREW PRAM, the running time is $2\lceil \log n \rceil$

If we assume a fixed number of processors p, with $n > p$. Each processor first sums an n/p section of the vector to generate a processor sum, the tree technique is then used to prescan the processor sums. The results of the prescan of the processor sums are used as an offset for each processor to prescan within its n/p section (see Figure **??**). The time complexity of the algorithm is:

$$T_S(n,p) = 2(\lceil n/p \rceil + \lceil \log p \rceil) = O(n/p + \log p)$$

$$[\underbrace{4 \quad 7 \quad 1}_{\text{processor 0}} \quad \underbrace{0 \quad 5 \quad 2}_{\text{processor 1}} \quad \underbrace{6 \quad 4 \quad 8}_{\text{processor 2}} \quad \underbrace{1 \quad 9 \quad 5}_{\text{processor 3}}]$$

$$\begin{array}{lcl} \text{Sum} & = & [12 \quad 7 \quad 18 \quad 15] \\ \text{+-prescan} & = & [0 \quad 12 \quad 19 \quad 37] \end{array}$$

$$[\underbrace{0 \quad 4 \quad 11}_{\text{processor 0}} \quad \underbrace{12 \quad 12 \quad 17}_{\text{processor 1}} \quad \underbrace{19 \quad 25 \quad 29}_{\text{processor 2}} \quad \underbrace{37 \quad 38 \quad 47}_{\text{processor 3}}]$$

FIGURE 1.6
A +-prescan with more elements than processors.

Figure 6

3 Line-of-Sight and Radix-Sort

Line-of-sight: given a terrain map in the form of a grid of altitudes and an observation point X on the grid, find which points are visible along a ray originating at the observation point. A point on a ray is visible if and only if no other point between it and the\ observation point has a greater vertical angle.

```
procedure line-of-sight(altitude)
  in parallel for each index i
    angle[i] ← arctan(scale × (altitude[i] - altitude[0])/ i)
  max-previous-angle ← max-prescan(angle)
  in parallel for each index i
    if (angle[i] > max-previous-angle[i])
      result[i] ← "visible"
    else
      result[i] ← not "visible"
```

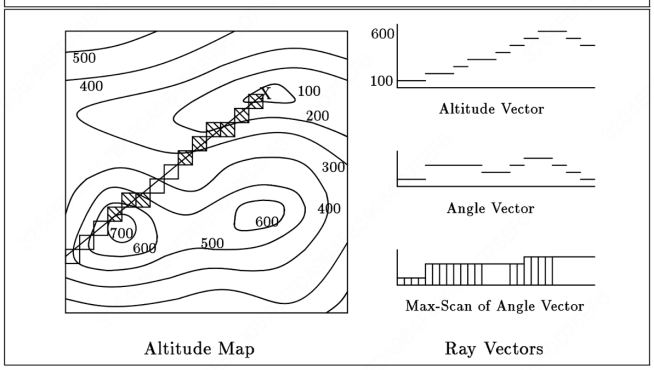


FIGURE 1.7
The line-of-sight algorithm for a single ray. The X marks the observation point. The visible points are shaded. A point on the ray is visible if no previous point has a greater angle.

Figure 7

To find if any previous point has a greater angle, the altitude of each point along the ray is placed in a vector (the **altitude vector**). These altitudes are then converted to angles and placed in the **angle vector**. A prescan using the operator maximum (max-prescan) is then executed on the **angle vector**, which returns to each point the maximum previous angle. To test for visibility, each point only needs to compare its angle to the result of the max-prescan.

For n points on a ray, the complexity of the algorithm is $T_S(n,p) = O(n/p + \log n)$ on an EREW PRAM.

Now consider *radix sort* algorithm. The algorithm loops over the bits of the keys, starting at the lowest bit, executing a split operation on each iteration (assume all the keys have the same number of bits). The

```
procedure split-radix-sort(A, number-of-bits)
  for i from 0 to (number-of-bits − 1)
    A ← split(A, A⟨i⟩)
```

A	=	[5	7	3	1	4	2	7	2]
A⟨0⟩	=	[1	1	1	1	0	0	1	0]
A ← split(A, A⟨0⟩)	=	[4	2	2	5	7	3	1	7]
A⟨1⟩	=	[0	1	1	0	1	1	0	1]
A ← split(A, A⟨1⟩)	=	[4	5	1	2	2	7	3	7]
A⟨2⟩	=	[1	1	0	0	0	1	0	1]
A ← split(A, A⟨2⟩)	=	[1	2	2	3	4	5	7	7]

FIGURE 1.8
An example of the split radix sort on a vector containing three bit values. The A⟨n⟩ notation signifies extracting the nth bit of each element of the vector A. The split operation packs elements with a 0 flag to the bottom and with a 1 flag to the top.

Figure 8

We now consider how to

```
procedure split(A, Flags)
  I-down ← +-prescan(not(Flags))
  I-up   ← n - +-scan(reverse-order(Flags))
  in parallel for each index i
    if (Flags[i])
      Index[i] ← I-up[i]
    else
      Index[i] ← I-down[i]
  result ← permute(A, Index)
```

A	=	[5	7	3	1	4	2	7	2]
Flags	=	[1	1	1	1	0	0	1	0]
I-down	=	[0	0	0	0	$\boxed{0}$	$\boxed{1}$	2	$\boxed{2}$]
I-up	=	[$\boxed{3}$	$\boxed{4}$	$\boxed{5}$	$\boxed{6}$	6	6	$\boxed{7}$	7]
Index	=	[3	4	5	6	0	1	7	2]
permute(A, Index)	=	[4	2	2	5	7	3	1	7]

FIGURE 1.9
The split operation packs the elements with a 0 in the corresponding flag position to the bottom of a vector, and packs the elements with a 1 to the top of the same vector. The permute writes each element of `A` to the index specified by the corresponding position in `Index`.

Figure 9

4 References